

Video Indexing and Retrieval

Hoa Dinh Nguyen, PhD.

hoand@ptit.edu.vn

Lecture Overview: Video Indexing and Retrieval

- ▶ Video retrieval approaches: metadata, text, audio, frame-based CBVR, segmentation
- ▶ Video segmentation: shot detection, gradual transitions, error reduction
- ▶ Representative frames and keyframe indexing
- ▶ Motion-based and entity-based retrieval
- ▶ Hybrid multimodal video IR architecture
- ▶ Prerequisites: Lecture 3 (video compression), 8 (text IR), 10 (audio), 11 (image CBIR)



Lecture Overview

- ▶ Video indexing approaches: metadata, text with timestamps, audio track, content-based frames
- ▶ Segmentation-based retrieval: partition video into shots/scenes → index keyframes
- ▶ Shot detection: frame difference, histogram difference, dual-threshold for gradual cuts
- ▶ Error reduction: optical flow, color normalization (illumination invariance)
- ▶ Representative frame selection: first frame, average frame, average histogram match
- ▶ Motion-based indexing: optical flow, motion vectors, camera pan/tilt detection
- ▶ Entity-based and combined multimodal retrieval

Video Indexing and Retrieval

- ▶ Based on common attributes (title, author name, production date, topic,...): same as regular database systems (relational data model).
- ▶ Text-based (subtitles, descriptions,...) with timestamps. Retrieval based on IR system
- ▶ Based on the audio data that accompanies the video, assisted by the corresponding timestamp. Similar to audio data retrieval.
- ▶ Content-based:
 - ▶ Video is a sequence of still images. The query process is based on the image query. Cons: loss of temporal relationship between frames, may have to process too many images.
 - ▶ Split the video file into small segments containing similar frames. Query based on the representative image for each video segment.
- ▶ Combination Method.

Content-Based Video Retrieval (CBVR) Framework

- ▶ Video = time-ordered frame sequence + synchronized audio + text tracks (subtitles)
- ▶ Approaches:
 - ▶ Metadata: title, author, date (relational DB)
 - ▶ Text IR: subtitles/transcripts with timestamps (Lecture 8)
 - ▶ Audio track: speech/music indexing (Lecture 10)
 - ▶ Frame CBVR: treat each frame as image — ignores temporal structure, costly
 - ▶ Segmentation + keyframes: detect shots → index representative frames — standard CBVR
 - ▶ Combination/hybrid: best accuracy for real systems



Indexing and Retrieval based on Segmentation

- ▶ Video is a continuous sequence of still images (frames)
- ▶ Videos can be broken down into segments consisting of a series of consecutive images that are highly related to each other:
 - ▶ Description of the same scene
 - ▶ Contains information about a camera operation
 - ▶ Contains information about a specific action or event of a certain entity
 - ▶ Frames are selected as a unique tool for indexing and retrieval.

Video Segments and Temporal Structure

- ▶ Shot: continuous frames from one camera take (cut separates shots)
- ▶ Scene: semantically related group of shots (same location/event)
- ▶ Segment types: same scene | camera operation (pan/zoom) | specific action/event
- ▶ Temporal redundancy: consecutive frames highly correlated
- ▶ Indexing unit: segment rather than every frame → 100×–1000× storage reduction
- ▶ MMDB entry: $\text{video}_{\text{oid}} + [(\text{seg}_{\text{id}}, t_{\text{start}}, t_{\text{end}}, \text{keyframe}_{\text{oid}}, \text{features} \dots)]$



Indexing and Retrieval based on Segmentation

- ▶ Key steps:

- ▶ Video segmentation / shot detection / partition
- ▶ Index each individual video segment: define the representative frames for each segment, apply indexing and retrieval methods for images to these frames.
- ▶ The retrieval process is based on a similarity comparison between the query data (image or any information about a certain frame) and the representative frames of the video segments.

Segmentation-Based Indexing Pipeline

- ▶ Step 1 - Shot detection: partition full video into segments at boundary frames
- ▶ Step 2 - Keyframe selection: pick 1–N representative frames per segment
- ▶ Step 3 - Feature extraction: color histogram, edge, CNN embedding per keyframe
- ▶ Step 4 - Index build: store feature vectors + timecodes in video segment table
- ▶ Step 5 - Query: compare query frame/sketch against keyframe index → return segments
- ▶ Step 6 - Playback: use $t_{\text{start}}/t_{\text{end}}$ to jump to matching segment in video player



Video segmentation / shot detection

- ▶ Frames in the same segment (shots) are often highly similar in some way
- ▶ If there is a difference between two consecutive frames (beyond a given threshold), then the two frames belong to two different segments (shots)
- ▶ Example:
 - ▶ Switching scenes between cameras (sudden transition)
 - ▶ Operations on cameras: entering or out-going one scene... (gradual transition)

Video segmentation

► Basic techniques:

- Find the sum of all the different attributes between two consecutive frames. Comparison with the threshold.
- Find the difference between the color histograms of two frames, comparing with a threshold.

$$SD_i = \sum_j |H_i(j) - H_{i+1}(j)| \text{ or } SD_i = \sqrt{\sum_j (H_i(j) - H_{i+1}(j))^2}$$

- Based on χ^2 test

$$SD_i = \sum_j \frac{(H_i(j) - H_{i+1}(j))^2}{H_{i+1}(j)}$$

The problem is to choose the right threshold values.

Video segmentation

- ▶ Segmentation with gradual scene transition
 - ▶ Use two thresholds, one to catch the sudden scene transition, one (lower value) threshold to catch the gradual scene transition.
 - ▶ The high threshold T_1 is used to identify sudden scene transition
 - ▶ The low threshold T_2 is used to mark the possibility of a gradual scene transition. If the total difference between X consecutive frames exceeds the threshold T_1 , the scene transition is determined.
 - ▶ Typical: $T_2 \approx 0.3 \times T_1$; $X = 10\text{--}30$ frames depending on frame rate
 - ▶ Fade-to-black: histogram approaches zero $\rightarrow$ special case detection

Video segmentation

► Minimize Segmentation Errors:

- Distinguish scene transitions from zooming in, zooming out on the camera, or panning with the camera: using the "optical flow" attribute.
- Distinguish scene transitions from changes in film brightness (due to lighting effects, or due to changes in light sources in film): normalization of colour values

- Normalization of component color values:

$$R'_i = \frac{R_i}{\sqrt{\sum_{k=1}^N R_k^2}}, \quad G'_i = \frac{G_i}{\sqrt{\sum_{k=1}^N G_k^2}}, \quad B'_i = \frac{B_i}{\sqrt{\sum_{k=1}^N B_k^2}}$$

- Conversion to Color values:

$$r_i = \frac{R'_i}{R'_i + G'_i + B'_i}, \quad g_i = \frac{G'_i}{R'_i + G'_i + B'_i}$$

- Construct color histograms of **r** and **g**.

Optical Flow and Color Normalization

- ▶ Problem: camera pan/zoom causes large pixel differences without shot change
- ▶ Optical flow: estimate motion field between frames; uniform flow → camera motion, not cut
- ▶ Shot cut: flow field discontinuous or magnitude spike inconsistent with pan model
- ▶ Illumination change: flash or lighting shift triggers false boundary
- ▶ Color normalization: chromaticity $r = R/(R+G+B)$, $g = G/(R+G+B)$ — removes brightness scale
- ▶ Index rg-histograms instead of RGB for shot detection under lighting variation

Video indexing and retrieval based on segmentation

- ▶ Each video segment is represented by one or a few frames
- ▶ Frames are treated as still images, video indexing and querying the same as for images
- ▶ During the query process, the similarity comparison is made between the query content and the representative frames.
- ▶ There are many ways to select representative frames for each video segment.

Keyframe Index Structure in MMDB

- ▶ VideoSegment table: seg_{id} , $\text{video}_{\text{oid}}$, t_{start} , t_{end} , $\text{shot}_{\text{type}}$, duration
- ▶ Keyframe table: kf_{id} , seg_{id} , $\text{frame}_{\text{number}}$, t_{offset} , $\text{thumbnail}_{\text{blob_oid}}$
- ▶ FeatureVector table: kf_{id} , $\text{hist}_{\text{vector}}$, $\text{edge}_{\text{hist}}$, $\text{cnn}_{\text{embedding}}$ (optional)
- ▶ TextTrack table: $\text{video}_{\text{oid}}$, t_{start} , t_{end} , $\text{transcript}_{\text{text}}$ (from ASR/subtitles)
- ▶ Query returns: ($\text{video}_{\text{oid}}$, seg_{id} , t_{start} , t_{end} , $\text{similarity}_{\text{score}}$)
- ▶ Player uses t_{start} for direct seek — critical UX for video search



Video indexing and retrieval based on representative frames

- ▶ Ways to select frames that represent each video segment
 - ▶ Select one single frame that represents each video segment. Cons: loss of information about the length of time as well as the change in content of the video.
 - ▶ Choose multiple frames to represent the video segment, the number of frames is based on the length of the segment, usually one frame is selected every second of the video.
 - ▶ Each video segment is broken down into pieces containing different content/scenes based on the vector of motion, optical flow, or the difference between successive frames. One representative frame is selected for each video piece.

Video indexing and retrieval based on representative frames

- ▶ Ways to select frames that represent each video segment
 - ▶ First frame: simple; misses action peak; good for static scenes
 - ▶ One frame per second: captures length; redundant for short shots
 - ▶ Average frame (per-pixel mean color): synthetic; pick closest real frame
 - ▶ Average histogram match: illumination-robust; pick frame with min L2 to mean histogram
 - ▶ Motion-based sub-segmentation: split shot by optical flow change → keyframe per sub-shot
 - ▶ Trade-off: more keyframes → better recall, larger index; typical: 1–3 keyframes per shot

Video indexing and retrieval

- ▶ Indexing and retrieval based on motion information: obtained from optical flow, or motion vectors.
 - ▶ The content of the motion. For example, dialogue videos have less moving content than action movies.
 - ▶ Homogeneity of motion: representation of continuity of the motion
 - ▶ Panning movement (horizontal)
 - ▶ Tilt (vertical) movement
 - ✓ Frames are indexed and queried based on both content and motion information

Motion-Based Video Indexing

- ▶ Motion magnitude: average optical flow magnitude per segment — action vs dialogue
- ▶ Motion homogeneity: variance of flow directions — smooth pan vs chaotic action
- ▶ Camera motion classification: pan (horizontal flow), tilt (vertical), zoom (radial flow)
- ▶ MPEG motion vectors: available in compressed bitstream without full decode
- ▶ MPEG-7 MotionActivity, CameraMotion descriptors — standardized motion metadata
- ▶ Query examples: 'high-action clips', 'static interview shots', 'panning landscape shots'



Video indexing and retrieval

- ▶ Indexing and retrieval based on entities
 - ▶ Entities in a video can be identified based on pixels moving between successive frames
 - ▶ Entities that can be used for video segmentation and video queries
- ▶ Video indexing and retrieval based on metadata using a common database system
- ▶ Video indexing and retrieval based on text content using IR systems
- ▶ Combined methods.

Entity-Based and Combined Retrieval

- ▶ Entity tracking: moving pixel regions across frames → object trajectory metadata
- ▶ Entities enable: 'find all segments containing person X', object re-identification
- ▶ Combined/hybrid retrieval:
 - ▶ metadata filter → text keyword search → audio classification → CBVR re-rank
- ▶ Example: 'find news clips about election' = text IR + face entity + metadata date filter
- ▶ Modern systems: multimodal embeddings (video-text-audio joint vectors) for single-query search



Summary — Video Indexing and Retrieval

- ▶ Video IR extends image/audio/text IR with temporal segmentation and timecoded indexes
- ▶ Shot detection is the foundation — dual threshold handles hard and gradual cuts
- ▶ Keyframe + CBIR enables visual query-by-example at segment granularity
- ▶ Motion and entity features add action, camera, and object-level search
- ▶ Hybrid multimodal architecture is standard for production video search systems
- ▶ Next: deep video embeddings (3D CNN, Video-LLM), real-time streaming index update

